

Exercises 32.1-1 to 32.1-4

Questions and Solutions One by One

Naive String-Matching Algorithm

1 Exercise 32.1-1

Question 32.1-1

Show the comparisons the naive string matcher makes for the pattern

$$P = 0001$$

in the text

$$T = 000010001010001.$$

Solution

We have

$$P = 0001, \quad m = 4$$

and

$$T = 000010001010001, \quad n = 15.$$

So the possible shifts are

$$s = 0, 1, 2, \dots, 11.$$

At each shift, the naive matcher compares from left to right and stops at the first mismatch.

Shift s	Text window	Comparisons made	Count
0	0000	$0 = 0, 0 = 0, 0 = 0, 1 \neq 0$	4
1	0001	$0 = 0, 0 = 0, 0 = 0, 1 = 1$	4
2	0010	$0 = 0, 0 = 0, 0 \neq 1$	3
3	0100	$0 = 0, 0 \neq 1$	2
4	1000	$0 \neq 1$	1
5	0001	$0 = 0, 0 = 0, 0 = 0, 1 = 1$	4
6	0010	$0 = 0, 0 = 0, 0 \neq 1$	3
7	0101	$0 = 0, 0 \neq 1$	2
8	1010	$0 \neq 1$	1
9	0100	$0 = 0, 0 \neq 1$	2
10	1000	$0 \neq 1$	1
11	0001	$0 = 0, 0 = 0, 0 = 0, 1 = 1$	4

Therefore, the valid shifts are

$$s = 1, 5, 11.$$

Total comparisons:

$$4 + 4 + 3 + 2 + 1 + 4 + 3 + 2 + 1 + 2 + 1 + 4 = \boxed{31}.$$

2 Exercise 32.1-2

Question 32.1-2

Suppose that all characters in the pattern P are different. Show how to accelerate NAIVE-STRING-MATCHER to run in time $\mathcal{O}(n)$ on an n -character text T .

Solution

Assume all characters of the pattern are distinct.

Suppose at shift s , the first q characters match:

$$P[1] = T[s + 1], P[2] = T[s + 2], \dots, P[q] = T[s + q],$$

but the next comparison fails:

$$P[q + 1] \neq T[s + q + 1].$$

Because all characters in P are different, the matched text characters

$$T[s + 1], T[s + 2], \dots, T[s + q]$$

correspond to

$$P[1], P[2], \dots, P[q],$$

which are all distinct.

Now any shift

$$s + 1, s + 2, \dots, s + q - 1$$

cannot be valid. The reason is that at any such shift, the first character $P[1]$ would align with one of the already matched characters $P[2], P[3], \dots, P[q]$, and none of them equals $P[1]$.

So after matching q characters and then failing, we can shift by q directly.

Improved Rule

- If no character matched, shift by 1.
- If $q > 0$ characters matched before mismatch, shift by q .

Algorithm

DISTINCT-NAIVE-MATCHER(T, P)

```
1  n = |T|, m = |P|
2  s = 0
3  while s <= n - m
4      q = 0
5      while q < m and P[q+1] == T[s+q+1]
6          q = q + 1
7      if q == m
8          print "Pattern occurs with shift", s
9          s = s + m
10     else if q == 0
11         s = s + 1
12     else
13         s = s + q
```

Running Time

Each time the algorithm makes successful comparisons, it later skips forward by exactly that many positions. So the total amount of work is linear in the text length.

Hence the running time is

$$\mathcal{O}(n).$$

3 Exercise 32.1-3

Question 32.1-3

Suppose that pattern P and text T are randomly chosen strings of length m and n , respectively, from the d -ary alphabet

$$\Sigma_d = \{0, 1, \dots, d-1\}, \quad d \geq 2.$$

Show that the expected number of character-to-character comparisons made by the implicit loop in line 4 of the naive algorithm is

$$(n - m + 1) \frac{1 - d^{-m}}{1 - d^{-1}} \leq 2(n - m + 1).$$

Solution

There are

$$n - m + 1$$

possible shifts.

Let C be the number of comparisons made at one fixed shift.

The i -th comparison happens only if the first $i-1$ characters match. Since the strings are random and independent,

$$\mathbb{P}(\text{first } i-1 \text{ characters match}) = d^{-(i-1)}.$$

So

$$\mathbb{P}(C \geq i) = d^{-(i-1)}, \quad i = 1, 2, \dots, m.$$

Using

$$\mathbb{E}[C] = \sum_{i=1}^m \mathbb{P}(C \geq i),$$

we get

$$\mathbb{E}[C] = \sum_{i=1}^m d^{-(i-1)} = \sum_{k=0}^{m-1} d^{-k}.$$

This is a geometric series:

$$\mathbb{E}[C] = \frac{1 - d^{-m}}{1 - d^{-1}}.$$

Since there are $n - m + 1$ shifts, the expected total number of comparisons is

$$(n - m + 1) \frac{1 - d^{-m}}{1 - d^{-1}}.$$

Now show the upper bound. Since $d \geq 2$,

$$d^{-1} \leq \frac{1}{2}.$$

Therefore,

$$\frac{1}{1-d^{-1}} \leq \frac{1}{1-\frac{1}{2}} = 2.$$

Also,

$$1-d^{-m} \leq 1.$$

Hence,

$$\frac{1-d^{-m}}{1-d^{-1}} \leq 2.$$

Multiplying by $(n-m+1)$:

$$(n-m+1) \frac{1-d^{-m}}{1-d^{-1}} \leq 2(n-m+1).$$

Thus the naive string matcher is quite efficient on average for random strings.

4 Exercise 32.1-4

Question 32.1-4

Suppose we allow the pattern P to contain occurrences of a gap character

\

that can match an arbitrary string of characters, even a string of length 0.

Give a polynomial-time algorithm to determine whether such a pattern P occurs in a given text T , and analyze the running time.

Solution

Let the gap symbol \ match any substring, including the empty substring.

Step 1: Split the pattern into blocks

Split the pattern at each gap symbol:

$$P = X_1 \setminus X_2 \setminus \cdots \setminus X_k,$$

where each X_i contains no gap symbol.

Example:

ab\ba\c

becomes

$$X_1 = \text{ab}, \quad X_2 = \text{ba}, \quad X_3 = \text{c}.$$

If the pattern contains only gap symbols, then it always matches the text.

Step 2: Matching idea

The pattern matches the text if and only if the blocks

$$X_1, X_2, \dots, X_k$$

occur in the text in this order, with anything allowed between them.

So we just need to find:

$$X_1 \text{ then } X_2 \text{ then } \cdots \text{ then } X_k.$$

Step 3: Greedy algorithm

Search for the blocks from left to right, always taking the earliest possible occurrence.

```
GAP-MATCH(T, P)
1  Split P into nonempty blocks X1, X2, ..., Xk
2  if k == 0
3      return true
4  pos = 1
5  for i = 1 to k
6      Find the leftmost occurrence of Xi in T starting at or after pos
7      if no such occurrence exists
8          return false
9      pos = ending position of that occurrence + 1
10 return true
```

Why it works

Choosing the earliest occurrence of each block leaves the maximum remaining space for later blocks. So if any valid full match exists, this greedy strategy will also succeed.

Running Time

Let

$$|T| = n, \quad |P| = m.$$

If we search each block using any polynomial-time substring search method, then a simple upper bound is

$$\mathcal{O}(kn + m),$$

where k is the number of blocks.

Since $k \leq m$, this is polynomial. In particular,

$$\boxed{\mathcal{O}(mn)}$$

is a valid bound.

Exercises 32.2-1 to 32.2-4

1 Exercise 32.2-1

Question 32.2-1

Working modulo $q = 11$, how many spurious hits does the Rabin–Karp matcher encounter in the text

$$T = 3141592653589793$$

when looking for the pattern

$$P = 26?$$

Solution

We use:

$$P = 26, \quad q = 11.$$

The pattern hash is

$$p = 26 \pmod{11} = 4.$$

So every text window of length 2 whose value is congruent to 4 (mod 11) will be a **hit**. A hit is:

- a **true hit** if the window is really 26,
- a **spurious hit** otherwise.

Now list all length-2 windows of the text:

$$31, 14, 41, 15, 59, 92, 26, 65, 53, 35, 58, 89, 97, 79, 93.$$

Compute each modulo 11:

$$\begin{aligned} 31 &\equiv 9, & 14 &\equiv 3, & 41 &\equiv 8, & 15 &\equiv 4, \\ 59 &\equiv 4, & 92 &\equiv 4, & 26 &\equiv 4, & 65 &\equiv 10, \\ 53 &\equiv 9, & 35 &\equiv 2, & 58 &\equiv 3, & 89 &\equiv 1, \\ 97 &\equiv 9, & 79 &\equiv 2, & 93 &\equiv 5. \end{aligned}$$

So the windows whose hash equals the pattern hash 4 are:

$$15, 59, 92, 26.$$

Among these:

- 26 is the real match,
- 15, 59, 92 are not equal to 26, so they are spurious hits.

Hence the number of spurious hits is

$$\boxed{3.}$$

Final Answer

The Rabin–Karp matcher encounters

3 spurious hits

at the windows

15, 59, 92.

2 Exercise 32.2-2

Question 32.2-2

How would you extend the Rabin–Karp method to the problem of searching a text string for an occurrence of any one of a given set of k patterns? Start by assuming that all k patterns have the same length. Then generalize your solution to allow the patterns to have different lengths.

Solution

We solve the problem in two stages.

Case 1: All k patterns have the same length m

Suppose the patterns are:

$$P_1, P_2, \dots, P_k,$$

and each has length m .

Idea

Instead of comparing the text window hash with one pattern hash, we compare it with the set of all pattern hashes.

Method

1. Compute the hash of each pattern:

$$h_i = \text{hash}(P_i) \pmod{q}, \quad i = 1, 2, \dots, k.$$

2. Store these hashes in a hash table or dictionary.
3. Run Rabin–Karp on the text using windows of length m .
4. For each window hash t_s :
 - if t_s is not in the set of pattern hashes, reject immediately;
 - if t_s matches one or more stored hashes, verify the actual substring against the corresponding patterns.

Why it works

A real occurrence of some pattern must have the same hash as that pattern. So every real match is detected as a hit. Then direct verification removes spurious hits.

Time Complexity

- Preprocessing pattern hashes:

$$\mathcal{O}(km).$$

- Text scan with rolling hash:

$$\mathcal{O}(n - m + 1) = \mathcal{O}(n).$$

- Verification happens only when a hit occurs.

So the total expected running time is roughly

$$\boxed{\mathcal{O}(km + n)}$$

plus verification cost.

Case 2: Patterns have different lengths

Now suppose the pattern lengths are not all equal.

Let the distinct lengths be:

$$m_1, m_2, \dots, m_r.$$

Method

Group the patterns by length.

For each length m_j :

1. collect all patterns of that length,
2. compute their hashes,
3. run Rabin–Karp on the text using windows of length m_j .

So we perform one Rabin–Karp scan for each distinct length.

Why this works

Rabin–Karp requires a fixed window length. If patterns have different lengths, then each distinct length needs its own rolling-hash scan.

Time Complexity

If there are r distinct lengths, then:

$$\text{total scan time} = \mathcal{O}(rn),$$

plus preprocessing of all patterns:

$$\mathcal{O}\left(\sum_{i=1}^k |P_i|\right).$$

So a clean bound is:

$$\boxed{\mathcal{O}\left(rn + \sum_{i=1}^k |P_i|\right)}$$

plus verification cost.

Final Answer

Same length case: hash all k patterns, store the hashes, and compare each text window hash against this set.

Different lengths case: group patterns by length and run a separate Rabin–Karp scan for each distinct length.

This correctly finds occurrences of any one of the k patterns.

3 Exercise 32.2-3

Question 32.2-3

Show how to extend the Rabin–Karp method to handle the problem of looking for a given $m \times m$ pattern in an $n \times n$ array of characters. The pattern may be shifted vertically and horizontally, but it may not be rotated.

Solution

We want to search for a 2D pattern inside a 2D text.

Let:

$$P = \text{an } m \times m \text{ pattern,} \quad T = \text{an } n \times n \text{ text array.}$$

Main Idea

The ordinary Rabin–Karp algorithm hashes a 1D string. For 2D matching, we compute a **2D rolling hash**.

The basic idea is:

- first hash each row,
- then combine the row hashes into a single block hash.

So the $m \times m$ block gets a single hash value.

Step 1: Row Hashes

Treat each row as a radix- d string.

For each row of the pattern, compute its hash:

$$r_i = \text{hash of row } i, \quad i = 1, 2, \dots, m.$$

Similarly, for each row of the text, compute rolling hashes of every length- m horizontal window. So for each text row, we can compute the hash of every horizontal segment of length m in $\mathcal{O}(1)$ update time.

Step 2: Combine Rows into a Block Hash

Now each $m \times m$ block consists of m row-hashes stacked vertically.

Treat these row-hashes as digits in another radix system and combine them:

$$H = \text{hash}(r_1, r_2, \dots, r_m).$$

This gives one hash for the whole $m \times m$ block.

Step 3: Rolling in Two Directions

We use rolling updates in two ways:

- **Horizontal rolling:** update row-window hashes as we move right.
- **Vertical rolling:** update block hashes as we move down.

So once the first block hash is computed, every neighboring block can be updated efficiently.

Algorithm Outline

1. Compute the hash of each row of the pattern.
2. Combine these into one pattern-block hash.
3. For each text row, compute hashes of all horizontal windows of length m .
4. For each column position, combine m consecutive row hashes into a vertical block hash.
5. Compare each text-block hash with the pattern-block hash.
6. If the hash matches, verify the actual $m \times m$ block.

Why it works

A true occurrence of the pattern must produce the same 2D hash as the pattern. If the hash matches, direct checking confirms whether it is a real occurrence or a spurious hit.

Time Complexity

There are roughly

$$(n - m + 1)^2$$

possible positions for the $m \times m$ pattern.

Using rolling hashes, each block hash can be updated in constant time after preprocessing. Thus the total scan time is

$$\boxed{\mathcal{O}(n^2)}$$

plus verification cost.

Final Answer

Extend Rabin–Karp to two dimensions by:

- hashing rows first,
- then hashing sequences of row-hashes vertically.

This gives a rolling 2D hash for each $m \times m$ block. Compare block hashes with the pattern hash, and verify only on a hit.

4 Exercise 32.2-4

Question 32.2-4

Alice has a copy of a long n -bit file

$$A = \langle a_{n-1}, a_{n-2}, \dots, a_0 \rangle,$$

and Bob similarly has

$$B = \langle b_{n-1}, b_{n-2}, \dots, b_0 \rangle.$$

Alice and Bob wish to know if their files are identical. To avoid transmitting all of A or B , they choose a prime

$$q > 1000n$$

and randomly choose an integer

$$x \in \{0, 1, \dots, q-1\}.$$

Then Alice evaluates

$$A(x) = \left(\sum_{i=0}^{n-1} a_i x^i \right) \pmod{q}$$

and Bob similarly evaluates

$$B(x) = \left(\sum_{i=0}^{n-1} b_i x^i \right) \pmod{q}.$$

Prove that if $A \neq B$, there is at most one chance in 1000 that

$$A(x) = B(x),$$

whereas if the two files are the same, $A(x)$ is necessarily the same as $B(x)$.

Solution

We consider two cases.

Case 1: $A = B$

If the files are identical, then for every i ,

$$a_i = b_i.$$

So the two polynomials are exactly the same:

$$A(x) = B(x)$$

for every choice of x .

Hence if $A = B$, then necessarily

$$A(x) = B(x).$$

Case 2: $A \neq B$

If the files are different, then there exists at least one index i such that

$$a_i \neq b_i.$$

Define the difference polynomial:

$$D(x) = A(x) - B(x) \pmod{q}.$$

Then $D(x)$ is **not** the zero polynomial, because $A \neq B$.

Also, the degree of $D(x)$ is at most

$$n - 1.$$

Key Fact

A nonzero polynomial of degree at most $n - 1$ over a field can have at most

$$n - 1$$

roots modulo q .

So the equation

$$A(x) = B(x)$$

is equivalent to

$$D(x) = 0,$$

and this can happen for at most $n - 1$ values of x .

Probability Bound

The value x is chosen uniformly from

$$\{0, 1, \dots, q - 1\},$$

which has q possible values.

Therefore,

$$\mathbb{P}(A(x) = B(x)) \leq \frac{n - 1}{q}.$$

Since

$$q > 1000n,$$

we have

$$\frac{n - 1}{q} < \frac{n}{1000n} = \frac{1}{1000}.$$

So:

$$\boxed{\mathbb{P}(A(x) = B(x) \mid A \neq B) < \frac{1}{1000}.$$

Hence there is at most one chance in 1000 that two different files produce the same value.

Final Conclusion

- If $A = B$, then

$$\boxed{A(x) = B(x) \text{ for every } x.}$$

- If $A \neq B$, then

$$\boxed{\mathbb{P}(A(x) = B(x)) < \frac{1}{1000}.$$

So this is a correct randomized fingerprint test for equality of two files.

Exercises 32.3-1 to 32.3-5

String Matching with Finite Automata

Questions and solutions one by one

1 Exercise 32.3-1

Question 32.3-1

Construct the string-matching automaton for the pattern

$$P = \text{aabab}$$

and illustrate its operation on the text

$$T = \text{aaababaabaababaab}.$$

Solution

The pattern is

$$P = \text{aabab}, \quad m = 5.$$

So the states of the automaton are:

$$Q = \{0, 1, 2, 3, 4, 5\}.$$

State q means that the first q characters of the pattern currently match a suffix of the text scanned so far.

The only accepting state is:

$$\boxed{5}.$$

Step 1: Compute the transition function

We use:

$$\delta(q, a) = \sigma(P_q a),$$

where $\sigma(x)$ is the length of the longest prefix of P that is also a suffix of x .

Since the alphabet here is effectively

$$\Sigma = \{a, b\},$$

we compute $\delta(q, a)$ and $\delta(q, b)$ for each state.

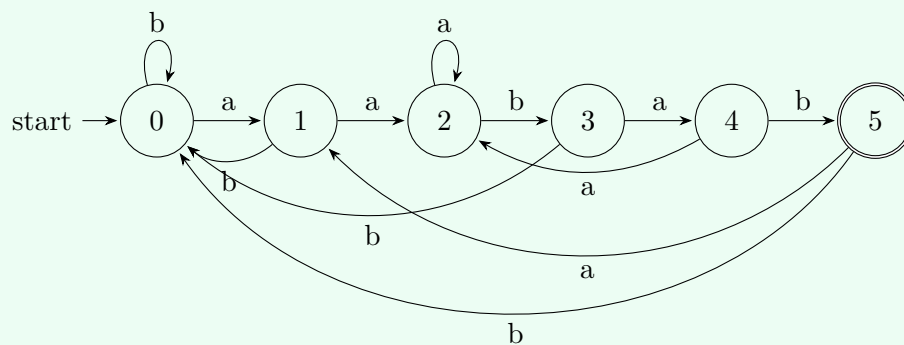
State q	$\delta(q, a)$	$\delta(q, b)$
0	1	0
1	2	0
2	2	3
3	4	0
4	2	5
5	1	0

Why these values?

A few examples:

- $\delta(0, a) = 1$: starting from nothing, reading a matches the first character.
- $\delta(2, b) = 3$: after matching aa , reading b gives suffix aab , so state 3.
- $\delta(4, b) = 5$: after matching $aaba$, reading b completes $aabab$.
- $\delta(5, a) = 1$: after a full match, reading a gives $aababa$; the longest prefix of P that is a suffix is only a , so the automaton returns to state 1.

State-transition diagram



Step 2: Run the automaton on the text

Text:

$T = \text{aaababaabaababab}$

We scan left to right.

i	$T[i]$	State after reading $T[i]$
1	a	1
2	a	2
3	a	2
4	b	3
5	a	4
6	b	5
7	a	1
8	a	2
9	b	3
10	a	4
11	a	2
12	b	3
13	a	4
14	b	5
15	a	1
16	a	2
17	b	3

Whenever the automaton reaches state 5, the pattern occurs.

So matches end at positions:

6, 14.

Therefore the shifts are:

$$6 - 5 = 1, \quad 14 - 5 = 9.$$

Final Answer

The string-matching automaton for $P = \text{aabab}$ has states 0 through 5, with transition table shown above. When run on

$$T = \text{aaababaabaababaab},$$

the automaton reaches the accepting state at positions

6, 14,

so the pattern occurs at shifts

1, 9.

2 Exercise 32.3-2

Question 32.3-2

Draw a state-transition diagram for a string-matching automaton for the pattern

$$P = \text{ababbabbababbababbabb}$$

over the alphabet

$$\Sigma = \{a, b\}.$$

Solution

We first note that

$$P = \text{ababbabbababbababbabb}$$

has length

$$m = 21.$$

Therefore the automaton has states

$$Q = \{0, 1, 2, \dots, 21\},$$

with start state 0 and accepting state

$$\boxed{21}.$$

Transition rule

For each state q and character $x \in \{a, b\}$,

$$\delta(q, x) = \sigma(P_q x),$$

where $\sigma(y)$ is the length of the longest prefix of P that is also a suffix of y .

Full transition table

The complete transition table is:

State q	$\delta(q, a)$	$\delta(q, b)$
0	1	0
1	1	2
2	3	0
3	1	4
4	3	5
5	6	0
6	1	7
7	3	8
8	9	0
9	1	10
10	11	0
11	1	12
12	3	13
13	14	0
14	1	15
15	16	8
16	1	17
17	3	18
18	19	0
19	1	20
20	3	21
21	9	0

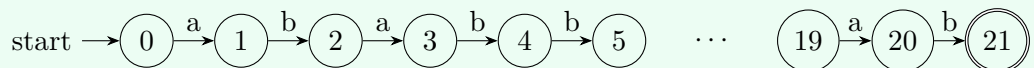
Main spine of the automaton

The forward matching edges follow the pattern exactly:

$$0 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{a} 3 \xrightarrow{b} 4 \xrightarrow{b} 5 \xrightarrow{a} 6 \xrightarrow{b} 7 \xrightarrow{b} 8 \xrightarrow{a} 9 \\ \xrightarrow{b} 10 \xrightarrow{a} 11 \xrightarrow{b} 12 \xrightarrow{b} 13 \xrightarrow{a} 14 \xrightarrow{b} 15 \xrightarrow{b} 16 \xrightarrow{a} 17 \xrightarrow{b} 18 \xrightarrow{b} 19 \xrightarrow{a} 20 \xrightarrow{b} 21.$$

Compact state-transition diagram

Because the full diagram with all fallback edges is very wide, a compact diagram is shown below together with the exact transition table above.



How to read the fallback transitions

All non-spine transitions are obtained from the table. For example:

- $\delta(5, b) = 0$: from state 5, reading b destroys the current match and sends the automaton back to state 0.

- $\delta(15, b) = 8$: from state 15, reading b does not go to 0; instead it falls back to state 8, because the resulting suffix matches the first 8 characters of the pattern.
- $\delta(21, a) = 9$: after a full match, reading a leaves a suffix that matches the first 9 characters of the pattern, so the automaton continues in state 9.

Final Answer

Thus the required string-matching automaton for

$$P = \text{ababbabbababbababbabb}$$

over $\Sigma = \{a, b\}$ has:

- states 0 through 21,
- start state 0,
- accepting state 21,
- transition function exactly as given in the full table above.

That table completely determines the full state-transition diagram.

3 Exercise 32.3-3

Question 32.3-3

We call a pattern P **nonoverlappable** if $P_k \supseteq P_q$ implies $k = 0$ or $k = q$. Describe the state-transition diagram of the string-matching automaton for a nonoverlappable pattern.

Solution

A pattern is **nonoverlappable** if no proper prefix of the pattern is also a suffix of any prefix of the pattern, except the trivial empty prefix.

That means:

$$P_k \supseteq P_q \implies k = 0 \text{ or } k = q.$$

What this means for the automaton

Normally, after a mismatch, the automaton may fall back to some smaller state $k > 0$. But for a nonoverlappable pattern, there are no useful proper prefix-suffix overlaps. So after a mismatch, the automaton almost always falls back directly to state 0.

Structure of the automaton

Suppose the pattern length is m . Then:

- states are $0, 1, 2, \dots, m$,
- the main spine is:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow m,$$

following the characters of the pattern,

- every mismatch transition from state q goes to state 0,
- except possibly when the mismatch character equals the first character of the pattern, in which case the automaton may go to state 1.

Why?

Since there is no nontrivial overlap, once the current match fails, the matched suffix cannot help continue the next match.

So the machine has no meaningful fallback state other than:

0

or occasionally:

1

if the current input character can begin a fresh match.

Diagram description

The automaton looks like a simple straight chain:

$$0 \xrightarrow{P[1]} 1 \xrightarrow{P[2]} 2 \xrightarrow{P[3]} \dots \xrightarrow{P[m]} m$$

and almost all other transitions go back to 0.

Key Shape

For a nonoverlappable pattern, the automaton is almost a straight line with reset edges back to 0.

Final Answer

For a nonoverlappable pattern, the state-transition diagram consists of:

- a main spine $0 \rightarrow 1 \rightarrow \dots \rightarrow m$ for correct matching characters,
- mismatch transitions that reset directly to state 0,
- with possible transitions to state 1 when the mismatch character equals the first character of the pattern.

So the automaton is much simpler than in the overlapping case.

4 Exercise 32.3-4

Question 32.3-4

Given two patterns P and P' , describe how to construct a finite automaton that determines all occurrences of either pattern. Try to minimize the number of states in your automaton.

Solution

We want one automaton that accepts whenever the text ends with:

$$P \quad \text{or} \quad P'.$$

Basic idea

Instead of building two completely separate automata, we combine them.

A state should represent:

the longest suffix of the text scanned so far that is also a prefix of either P or P' .

So we should not keep duplicate states for prefixes that are identical in the two patterns.

Best construction idea: use merged prefix states

Collect all prefixes of P and all prefixes of P' , including the empty string.

Then merge equal prefixes into a single state.

So each state corresponds to a distinct string x such that:

$$x \text{ is a prefix of } P \text{ or } P'.$$

The accepting states are:

$$P \quad \text{and} \quad P'.$$

Transition rule

If the current state corresponds to string x , and the next input character is a , then move to the longest suffix of xa that is also a prefix of P or P' .

This is exactly the same idea as ordinary string-matching automata, except now the allowed prefixes come from either pattern.

Why this minimizes the number of states

If P and P' share common prefixes, we should reuse them instead of duplicating them.

For example, if:

$$P = \text{abab}, \quad P' = \text{abac},$$

then the prefixes

$$\varepsilon, \text{a}, \text{ab}, \text{aba}$$

are shared.

So a merged automaton is smaller than two separate automata.

Conceptual structure

This construction is equivalent to building a trie of the two patterns and then adding fallback transitions based on longest suffix-prefix matches.

Minimal-State Idea

Merge all common prefixes of the two patterns into shared states.

Final Answer

To construct a finite automaton for matching either P or P' :

1. Create one state for every distinct prefix of P or P' .
2. Merge identical prefixes into the same state.
3. Mark the states corresponding to P and P' as accepting.
4. For each state x and character a , transition to the longest suffix of xa that is also a prefix of P or P' .

This construction minimizes states by sharing all common prefixes.

5 Exercise 32.3-5

Question 32.3-5

Given a pattern P containing gap characters, show how to build a finite automaton that can find an occurrence of P in a text T in $O(n)$ matching time, where $n = |T|$.

Solution

Let the pattern contain a special gap symbol that can match any string, including the empty string.

For example:

$$P = X_1 \text{ gap } X_2 \text{ gap } \cdots \text{ gap } X_k,$$

where each X_i is an ordinary string without gap symbols.

Main idea

A match occurs if the text contains:

$$X_1 \text{ followed later by } X_2 \text{ followed later by } \cdots \text{ followed later by } X_k.$$

The gap can absorb any substring between two consecutive fixed blocks.

So the automaton must keep track of:

how many blocks have already been matched in order, and how much of the current block is matched so far.

Construction idea

For each block X_i , build the usual string-matching automaton for that block.

Then connect these automata in sequence:

- once the automaton for X_1 reaches its accepting state, the machine moves into the automaton for X_2 ,
- similarly from X_2 to X_3 , and so on.

Because the gap can match any substring, once block X_i is completed, the automaton remains ready to begin matching block X_{i+1} at any later position.

State meaning

A state can be represented as:

$$(i, q),$$

where:

- i tells which block we are currently matching,
- q tells how many characters of that block have been matched.

Once the last block X_k is fully matched, we accept.

Why matching time is $O(n)$

As the text is scanned left to right:

- each character causes one transition,
- the automaton never backs up in the text,
- all state changes are constant-time transitions.

So after preprocessing, the matching phase takes:

$$O(n).$$

Preprocessing

The preprocessing builds a finite automaton whose states encode:

- current block number,
- current matched prefix length inside that block.

This preprocessing depends only on the pattern, not on the text.

Final Answer

To handle gap characters in a finite automaton:

1. split the pattern into ordinary blocks

$$X_1, X_2, \dots, X_k,$$

2. build automaton states that track which block is active and how much of it is matched,
3. after completing one block, allow arbitrary text until the next block begins matching,
4. accept when the final block is completed.

Since the resulting automaton scans the text once from left to right, the matching time is

$$O(n).$$

Exercises 32.4-1 to 32.4-8

1 Exercise 32.4-1

Question 32.4-1

Compute the prefix function for the pattern

$$P = \text{ababbabbabbababbabb.}$$

Solution

Let

$$P = \text{ababbabbabbababbabb.}$$

Index the characters:

q	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$P[q]$	a	b	a	b	b	a	b	b	a	b	b	a	b	a	b	b	a	b	b

The prefix-function values are:

$$\pi[1] = 0$$

because a single character has no proper prefix-suffix.

Proceeding through the pattern, we obtain:

q	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$\pi[q]$	0	0	1	2	0	1	2	0	1	2	0	1	2	3	4	5	6	7	8

So the prefix function is:

$$\pi = [0, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 3, 4, 5, 6, 7, 8].$$

Final Answer

$$\pi = [0, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 3, 4, 5, 6, 7, 8].$$

2 Exercise 32.4-2

Question 32.4-2

Give an upper bound on the size of $\pi^*[q]$ as a function of q . Give an example to show that your bound is tight.

Solution

The chain

$$\pi^*[q] = \{\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots\},$$

strictly decreases, since

$$\pi[x] < x.$$

Starting below q , it can contain at most q values:

$$|\pi^*[q]| \leq q.$$

This is tight. For $P_q = \text{aaaaa}$ with $q = 5$,

$$\pi[5] = 4, \quad \pi[4] = 3, \quad \pi[3] = 2, \quad \pi[2] = 1, \quad \pi[1] = 0,$$

so

$$\pi^*[5] = \{4, 3, 2, 1, 0\},$$

whose size is $5 = q$.

Final Answer

$|\pi^*[q]| \leq q$, and the bound is tight for repeated-character patterns such as $\text{aaaa} \dots$.

3 Exercise 32.4-3

Question 32.4-3

Explain how to determine the occurrences of pattern P in the text T by examining the prefix function for the string PT (the concatenation of P and T).

Solution

Let $|P| = m$. Form $S = P\#T$, where $\#$ is a separator not in the alphabet, and compute the prefix function of S .

Whenever

$$\pi[i] = m,$$

the pattern P ends at position i of S . Since S starts with $P\#$, the corresponding shift in T is

$$i - 2m - 1$$

using 1-based indexing for S .

Final Answer

Compute π for $P\#T$. Every index i with $\pi[i] = |P|$ gives one occurrence of P in T .

4 Exercise 32.4-4

Question 32.4-4

Use an aggregate analysis to show that the running time of KMP-MATCHER is $\Theta(n)$.

Solution

The only suspicious part is the fallback loop:

```
while q > 0 and P[q+1] != T[i]
    q = pi[q]
```

During the whole scan:

- q increases at most n times, once per text character,
- each fallback strictly decreases q , since $\pi[q] < q$,
- q never becomes negative.

So the total number of fallbacks is also at most n . The outer loop has n iterations, hence the total work is $\Theta(n)$.

Final Answer

$\Theta(n)$, because q can increase at most n times and decrease at most n times.

5 Exercise 32.4-5

Question 32.4-5

Use a potential function to show that the running time of KMP-MATCHER is $\Theta(n)$.

Solution

Use the potential

$$\Phi = q,$$

where q is the number of currently matched pattern characters.

For one fallback $q \leftarrow \pi[q]$, the actual cost is 1, and the potential drops by at least 1. Thus its amortized cost is

$$1 + (\Phi_{\text{new}} - \Phi_{\text{old}}) = 1 + (\pi[q] - q) \leq 0.$$

For a successful match $q \leftarrow q + 1$, the amortized cost is at most

$$1 + 1 = 2.$$

There are at most n successful extensions and n outer-loop iterations, so the total amortized work is $O(n)$. Reading the text gives the lower bound $\Omega(n)$.

Final Answer

With potential $\Phi = q$, fallbacks pay for themselves by decreasing Φ . Hence KMP-MATCHER runs in $\Theta(n)$.

6 Exercise 32.4-6

Question 32.4-6

Show how to improve KMP-MATCHER by replacing the occurrence of π in line 7 (but not line 12) by π' , where π' is defined recursively for $q = 1, 2, \dots, m - 1$ by

$$\pi'[q] = \begin{cases} 0 & \text{if } \pi[q] = 0, \\ \pi'[\pi[q]] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] = P[q + 1], \\ \pi[q] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] \neq P[q + 1]. \end{cases}$$

Explain why the modified algorithm is correct, and in what sense it is an improvement.

Solution

The improved function π' skips fallback states that would fail on the same character. If

$$P[\pi[q] + 1] = P[q + 1],$$

then after a mismatch at state q , falling back to $\pi[q]$ would immediately mismatch again. So we jump farther:

$$\pi'[q] = \pi'[\pi[q]].$$

Otherwise the fallback state may be useful, so keep

$$\pi'[q] = \pi[q].$$

Replace only line 7 by:

```
q = pi'[q]
```

Keep line 12 unchanged:

```
q = pi[q]
```

π' preserves correctness because it skips only states that are guaranteed to fail immediately. Line 12 still uses π so overlapping matches are handled correctly.

Final Answer

Use π' only for mismatch fallback. The running time remains $\Theta(n)$, but the algorithm performs fewer redundant comparisons.

7 Exercise 32.4-7

Question 32.4-7

Give a linear-time algorithm to determine whether a text T is a cyclic rotation of another string T' . For example, **arc** and **car** are cyclic rotations of each other.

Solution

Two strings T and T' are cyclic rotations of each other if and only if:

- they have the same length,
- one appears as a substring of the other doubled.

1. If $|T| \neq |T'|$, return false.
2. Form $S = TT$.
3. Run KMP to test whether T' occurs in S .
4. If yes, return true; otherwise return false.

Example: $T = \text{arc}$, $TT = \text{arcarc}$, and $T' = \text{car}$ appears inside it. The running time is $O(n)$, because KMP searches a text of length $2n$.

Final Answer

Check whether T' occurs in TT using KMP. This solves the problem in $\boxed{O(n)}$ time.

8 Exercise 32.4-8

Question 32.4-8

Give an $O(m|\Sigma|)$ -time algorithm for computing the transition function δ for the string-matching automaton corresponding to a given pattern P . (Hint: Prove that $\delta(q, a) = \delta(\pi[q], a)$ if $q = m$ or $P[q + 1] \neq a$.)

Solution

First compute π in $\Theta(m)$. Then fill the transition table using:

$$\delta(q, a) = \begin{cases} q + 1, & \text{if } q < m \text{ and } P[q + 1] = a, \\ \delta(\pi[q], a), & \text{otherwise.} \end{cases}$$

COMPUTE-TRANSITION-FUNCTION-FAST(P , Σ)

```
1  m = |P|
2  compute pi[1..m]
3  for each a in Sigma
4      delta(0, a) = 0
5  delta(0, P[1]) = 1
6  for q = 1 to m
7      for each a in Sigma
8          if q < m and P[q+1] == a
9              delta(q, a) = q + 1
10         else
11             delta(q, a) = delta(pi[q], a)
12 return delta
```

If a extends the current match, move to $q + 1$. Otherwise, fall back to the largest border

state $\pi[q]$ and reuse its transition. There are $m + 1$ states and $|\Sigma|$ characters, so the table has $O(m|\Sigma|)$ entries.

Final Answer

Use the recurrence

$$\delta(q, a) = \begin{cases} q + 1, & \text{if } q < m \text{ and } P[q + 1] = a, \\ \delta(\pi[q], a), & \text{otherwise.} \end{cases}$$

After computing π , all entries are filled in $\boxed{O(m|\Sigma|)}$ time.

String Matching Review

Questions and Answers

Indexing Convention

For algorithm traces, text and pattern positions are shown with 0-based indexing.

Question 1

List the prefixes of the string $P = \text{aaabbaaa}$ that are also suffixes of P .

Answer 1

The prefixes that are also suffixes are

`a, aa, aaa, aaabbaaa.`

If only proper prefixes are required, remove the whole string:

`a, aa, aaa.`

Question 2

What is the longest proper prefix of `cgtacgttcgtacg` that is also a suffix?

Answer 2

The longest proper prefix-suffix is

`cgtacg.`

It appears at both ends:

`cgtacgttcgtacg.`

Question 3

Draw a figure illustrating the comparisons done by brute-force pattern matching for text `aaabaadaabaaa` and pattern `aabaaa`.

Answer 3

Let

$T = \text{aaabaadaabaaa}, \quad P = \text{aabaaa}.$

Shift	Comparison result	Outcome
0	$a = a, a = a, a \neq b$	fail
1	$a = a, a = a, b = b, a = a, a = a, d \neq a$	fail
2	$a = a, b \neq a$	fail
3	$b \neq a$	fail
4	$a = a, a = a, d \neq b$	fail
5	$a = a, d \neq a$	fail
6	$d \neq a$	fail
7	$a = a, a = a, b = b, a = a, a = a, a = a$	match

Therefore the pattern occurs at

shift 7.

The brute-force method performs 24 character comparisons.

Question 4

Repeat the previous matching example using the Knuth-Morris-Pratt algorithm, not counting the comparisons made to compute the failure function.

Answer 4

For $P = \text{aabaaa}$, the KMP prefix/failure table is

$$\pi = [0, 1, 0, 1, 2, 2].$$

Text index	$T[i]$	Main action	Matched length after step
0	a	extend	1
1	a	extend	2
2	a	fallback $2 \rightarrow 1$, then extend	2
3	b	extend	3
4	a	extend	4
5	a	extend	5
6	d	fallback $5 \rightarrow 2 \rightarrow 1 \rightarrow 0$	0
7	a	extend	1
8	a	extend	2
9	b	extend	3
10	a	extend	4
11	a	extend	5
12	a	extend to full match	2

The full match ends at text index 12, so the shift is

$$12 - 6 + 1 = 7.$$

KMP reports the pattern at shift 7.

Question 5

Compute a table representing the KMP failure function for the pattern string **cgtacgttcgtac**.

Answer 5

For $P = \text{cgtacgttcgtac}$, the prefix/failure function is:

q	0	1	2	3	4	5	6	7	8	9	10	11	12
$P[q]$	c	g	t	a	c	g	t	t	c	g	t	a	c
$\pi[q]$	0	0	0	0	1	2	3	0	1	2	3	4	5

$$\pi = [0, 0, 0, 0, 1, 2, 3, 0, 1, 2, 3, 4, 5].$$